CBD PROYECTO FITLOG API

Pablo Arrabal Hermoso
Miguel Regidor García

ÍNDICE

01. MOTIVACIÓN

02. STACK TECNOLÓGICO

03. UML

04. RESTRICCIONES

05. IMPLEMENTACIÓN

06. CONCLUSIONES

01

MOTIVACIÓN

¿Por qué una app de entrenamiento?

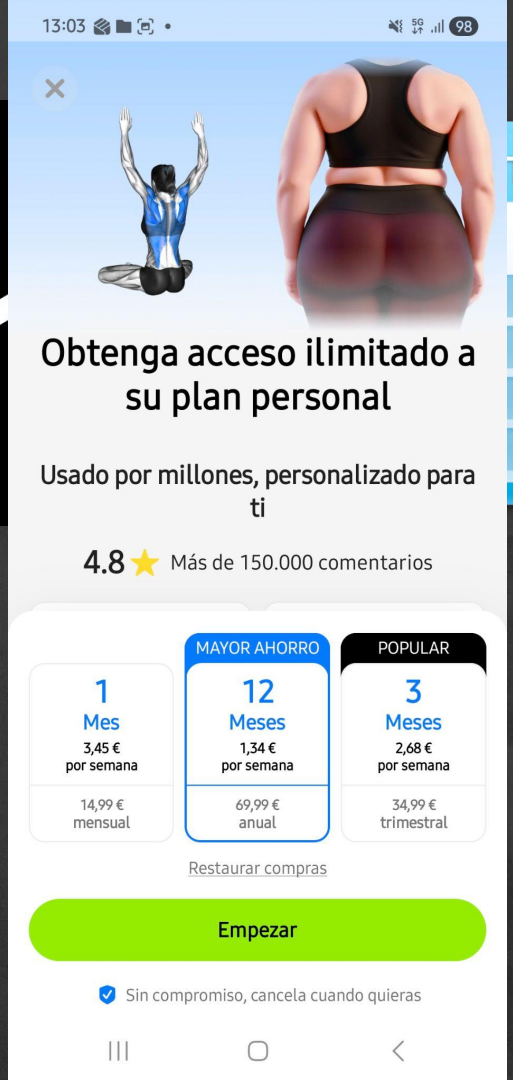
1. MOTIVACIÓN

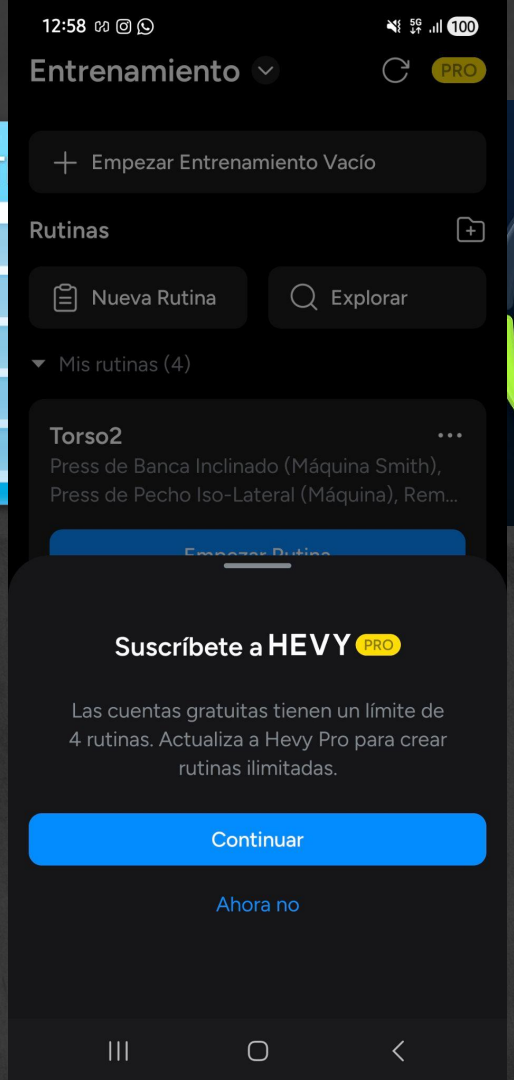
01

Usuarios habituales

02

Barreras de pago


13:03
98
Obtenga acceso ilimitado a su plan personal
Usado por millones, personalizado para ti
4.8 Más de 150.000 comentarios
MAYOR AHORRO
POPULAR
1 Mes 3,45 € por semana
14,99 € mensual
12 Meses 1,34 € por semana
69,99 € anual
3 Meses 2,68 € por semana
34,99 € trimestral
Restaurar compras
Empezar
Sin compromiso, cancela cuando quieras


12:58
100
Entrenamiento
PRO
Empezar Entrenamiento Vacío
Rutinas
Nueva Rutina
Explorar
Mis rutinas (4)
Torso2
Press de Banca Inclinado (Máquina Smith), Press de Pecho Iso-Lateral (Máquina), Rem...
Suscríbete a HEVY PRO
Las cuentas gratuitas tienen un límite de 4 rutinas. Actualiza a Hevy Pro para crear rutinas ilimitadas.
Continuar
Ahora no

2. STACK TECNOLÓGICO

MONGODB

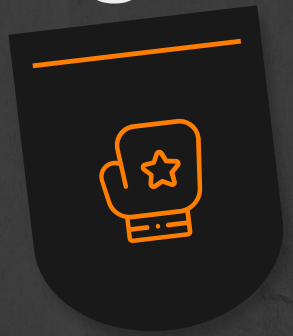
CON MONGOOSE

MongoDB®

NODE.JS

CON EXPRESS

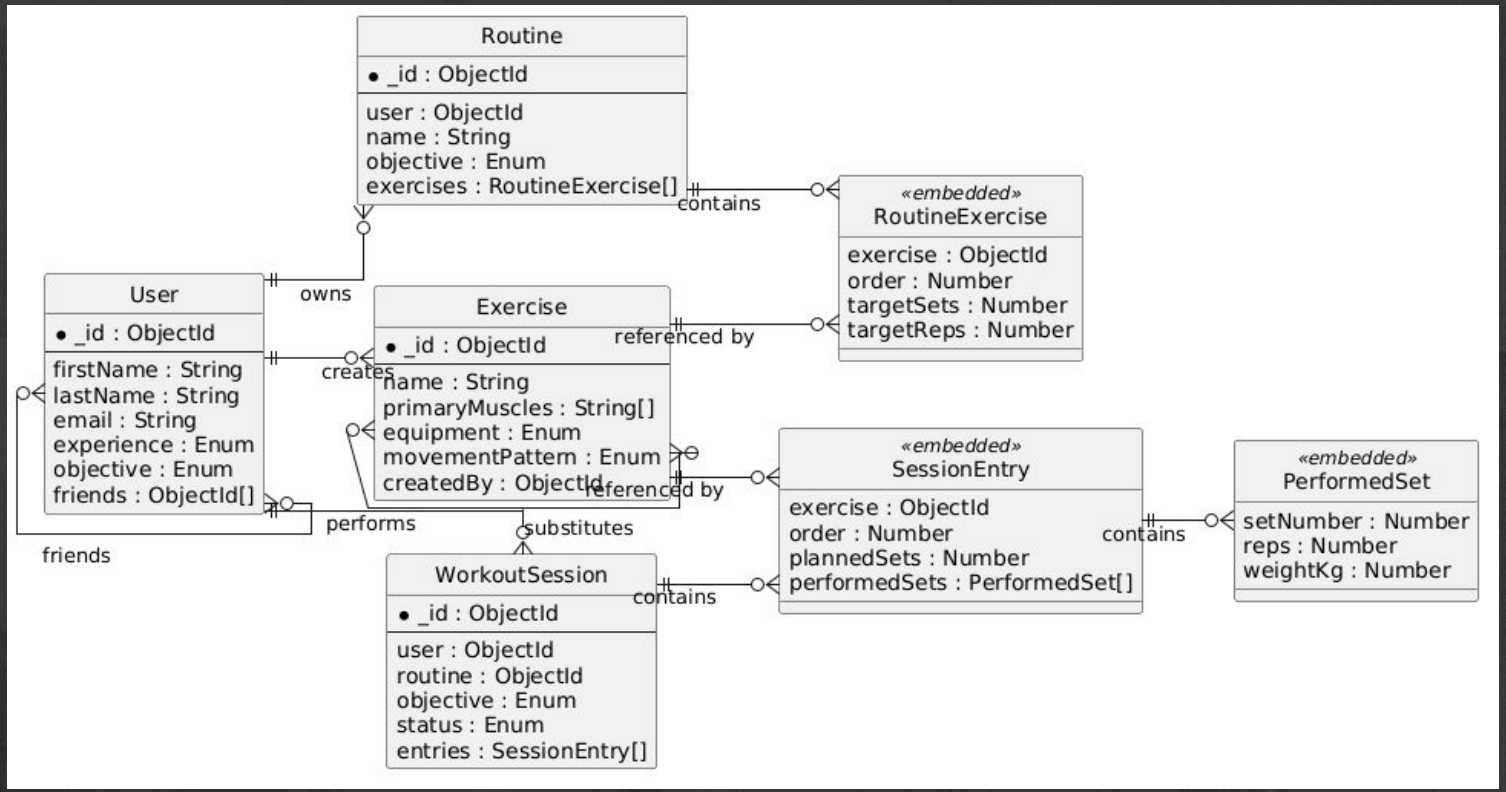
¿Por qué noSQL?

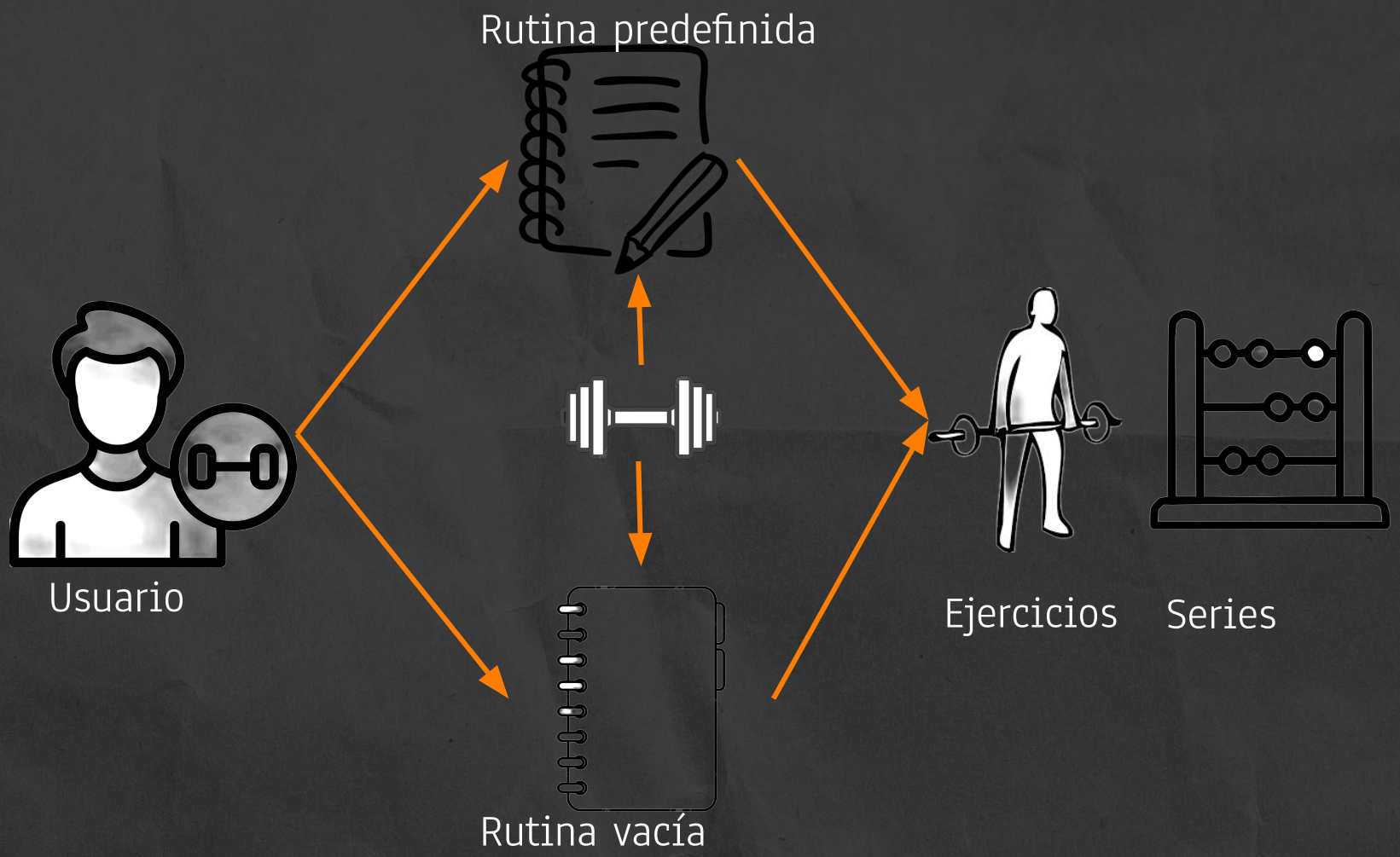


¿Series?
¿Tiempo?
¿Peso?

Heterogeneidad de los ejercicios

3.Modelo de datos




Rutina predefinida
Usuario
Ejercicios
Series
Rutina vacía

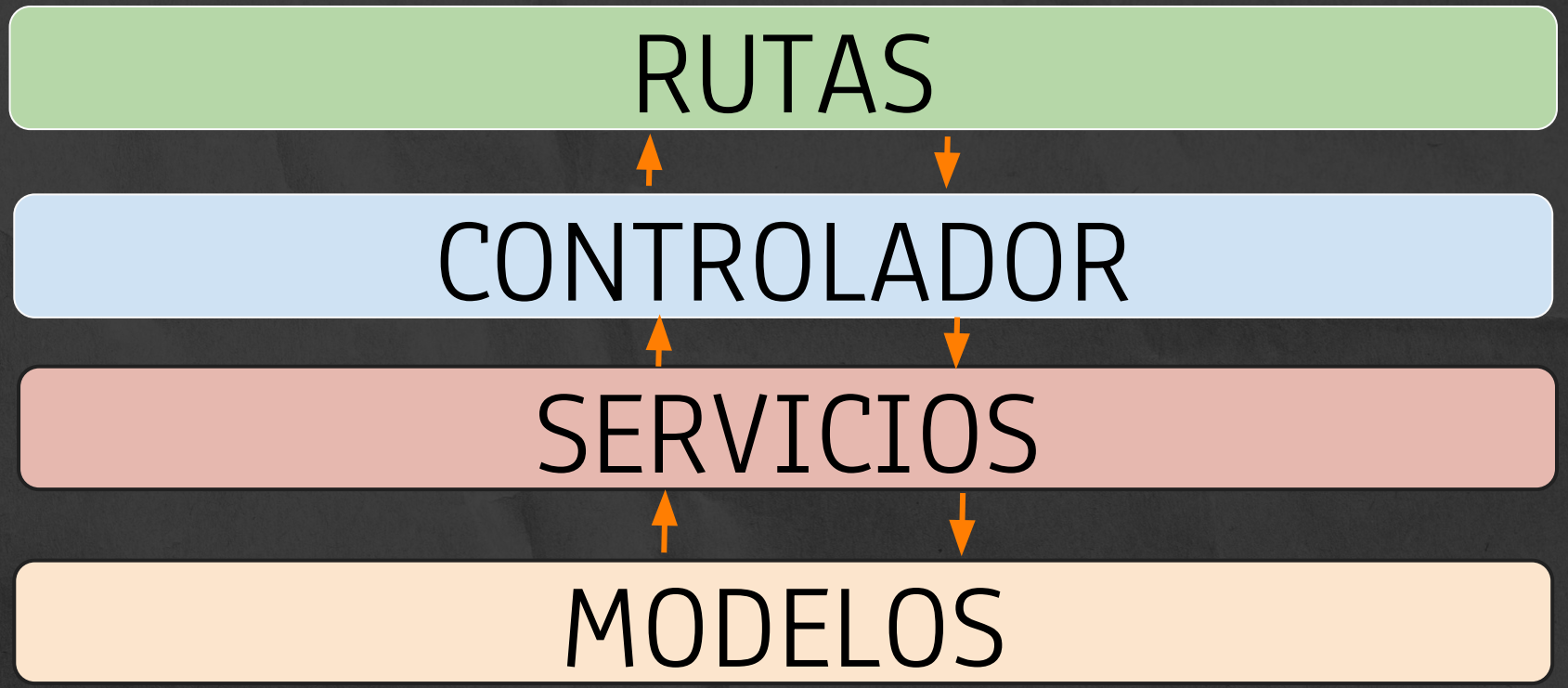
4.Restricciones

```
const exerciseQuerySchema = z.object({
  q: z.string().max(100).optional(),
  muscle: z.string().max(60).optional(),
  equipment: z.enum(['barbell', 'dumbbell', 'machine', 'cable', 'bodyweight', 'kettlebell', 'other']).optional(),
  limit: z.coerce.number().int().min(1).max(100).optional().default(25),
  page: z.coerce.number().int().min(1).optional().default(1)
});

const exerciseBodySchema = z.object({
  name: z.string().trim().min(2).max(100),
  description: z.string().max(1000).optional().default(''),
  primaryMuscles: z.array(z.string().min(1)).min(1),
  secondaryMuscles: z.array(z.string().min(1)).optional().default([]),
  equipment: z.enum(['barbell', 'dumbbell', 'machine', 'cable', 'bodyweight', 'kettlebell', 'other']).optional().default('other'),
  movementPattern: z.enum(['push', 'pull', 'squat', 'hinge', 'carry', 'core', 'other']).optional().default('other'),
  difficulty: z.enum(['easy', 'moderate', 'hard']).optional().default('moderate'),
  visibility: z.enum(['public', 'private']).optional().default('public'),
  substitutes: z.array(z.string().min(1)).optional().default([])
});
```

MONGOOSE

```
const routineExerciseSchema = z
  .object({
    exercise: z.string().min(1),
    order: z.number().int().positive(),
    targetSets: z.number().int().min(1).max(20),
    targetRepsMin: z.number().int().min(1).max(50),
    targetRepsMax: z.number().int().min(1).max(50),
    targetWeightKg: z.number().min(0).max(700).optional(),
    restSeconds: z.number().int().min(10).max(600).optional(),
    notes: z.string().max(300).optional()
  })
  .refine((data) => data.targetRepsMin <= data.targetRepsMax, {
    message: 'targetRepsMin cannot be greater than targetRepsMax',
    path: ['targetRepsMin']
  });

const createRoutineSchema = z.object({
  name: z.string().trim().min(2).max(100),
  objective: z.enum(['strength', 'hypertrophy', 'endurance', 'recomposition']).optional(),
  description: z.string().max(400).optional(),
  exercises: z.array(routineExerciseSchema).max(40).optional().default([])
});
```

5.Implementación



Modelos

{ "_id": "65f8a123e4b0a1a2b3c4d111",
"name": "Press de Banca con Barra",
"description": "Ejercicio compuesto fundamental para el desarrollo del torso. ",
"primaryMuscles": ["Chest"],
"secondaryMuscles": ["Triceps", "Front Deltoids"],
"equipment": "barbell",
"movementPattern": "push",
"difficulty": "moderate",
"visibility": "public",
"approvalStatus": "approved",
"substitutes": ["65f8a123e4b0a1a2b3c4d888",
"65f8a123e4b0a1a2b3c4d999"],
"isPublic": true,
...}

Ejercicio

{"_id": "65f8a1f5e4b0a1a2b3c4d505",
"name": "Push Day - Fuerza",
"objective": "strength",
"user": "65f8a1a1e4b0a1a2b3c4d501",
"exercises": [
{
"exercise": "65f8a123e4b0a1a2b3c4d111",
"order": 1,
"targetSets": 4,
"targetRepsMin": 5,
"targetRepsMax": 8,
"targetWeightKg": 50,
"restSeconds": 180
},
{
"exercise": "65f8a123e4b0a1a2b3c4d222"
"order": 2,
"targetSets": 3,
"targetRepsMin": 8,
"targetRepsMax": 12,
"targetWeightKg": 15,
"restSeconds": 90,
"notes": "Cuidar el rango de movimiento"
}
],
"isArchived": false,
"createdAt": "2024-03-20T10:00:00Z"}

Rutina

Funcionalidades

CAPA SOCIAL

CRUD de usuarios

CRUD de amigos

CAPA DE ENTRENOS

CRUD de ejercicios

CRUD de rutinas

CRUD de sesiones de entrenamiento

CAPA DE CONSULTA

READ de entrenos pasados

CONCEPTOS AVANZADOS

MONGODB + NODE.JS

Hooks

```
/**
 * Encripta la contraseña antes de guardar (insert o update)
 */
userSchema.pre('save', async function hashPassword(next) {
  if (!this.isModified('password')) {
    return next();
  }

  this.password = await bcrypt.hash(this.password, 12);
  return next();
});

/**
 * Encripta la contraseña para operaciones de actualización atómicas
 * Maneja: findOneAndUpdate, updateOne, updateMany, update
 */
userSchema.pre(['findOneAndUpdate', 'updateOne', 'updateMany', 'update'], async function (next) {
  const update = this.getUpdate();
  if (!update) return next();

  if (update.password) {
    update.password = await bcrypt.hash(update.password, 12);
    this.setUpdate(update);
  }
  else if (update.$set && update.$set.password) {
    update.$set.password = await bcrypt.hash(update.$set.password, 12);
    this.setUpdate(update);
  }

  return next();
});
```

Optimización

Propiedades Virtuales

Operaciones Atómicas

Operadores de Selección

Optimización

Propiedades Virtuales

Campos que no se guardan en la DB pero se calculan al vuelo para facilitar el acceso a datos relacionados.

```
userSchema.virtual('totalExercises', {
  ref: 'Exercise',
  localField: '_id',
  foreignField: 'createdBy',
  count: true
});

userSchema.virtual('activeRoutines', {
  ref: 'Routine',
  localField: '_id',
  foreignField: 'user',
  count: true
});

userSchema.virtual('friendsCount').get(function () {
  return this.friends ? this.friends.length : 0;
});
```

Optimización

Operaciones Atómicas

Garantizan que las actualizaciones se realicen de forma íntegra, evitando condiciones de carrera en entornos concurrentes.

```
const addFriend = async (userId, friendUsername) => {
  // Find friend by username
  const friend = await User.findOne({ username: friendUsername });
  if (!friend) {
    throw new ApiError(404, 'Friend not found');
  }
  const friendId = friend._id;

  if (String(friendId) === String(userId)) {
    throw new ApiError(400, 'You cannot add yourself as a friend');
  }

  const currentUser = await ensureUserExists(userId, 'User not found');


  await Promise.all([
    User.findByIdAndUpdate(
      userId,
      { $addToSet: { friends: friendId } },
      { new: true }
    ),
    User.findByIdAndUpdate(
      friendId,
      { $addToSet: { friends: userId } },
      { new: true }
    )
  ]);

  return { friendId };
};
```

Optimización

Operadores de Selección

Uso eficiente de proyecciones para recuperar únicamente la información necesaria de la base de datos.

```
const listSessions = async (userId) => {
  const sessions = await WorkoutSession.find({ user: userId })
    .sort({ startedAt: -1 })
    .populate('entries.exercise', 'name primaryMuscles')
    .populate('routine', 'name objective');

  return sessions;
};
```

```
const listUsers = async () => {
  const users = await User.find()
    .sort({ createdAt: -1 })
    .select(PUBLIC_USER_FIELDS);

  return users;
};
```

Protección

Validación con ZOD

Garantizamos la integridad total de los datos de entrada mediante esquemas de validación estrictos.

Middlewares de Defensa

Capas intermedias dedicadas a la protección y filtrado de información sensible en cada petición.

DEMO VISUAL

06

CONCLUSIONES

6.CONCLUSIONES

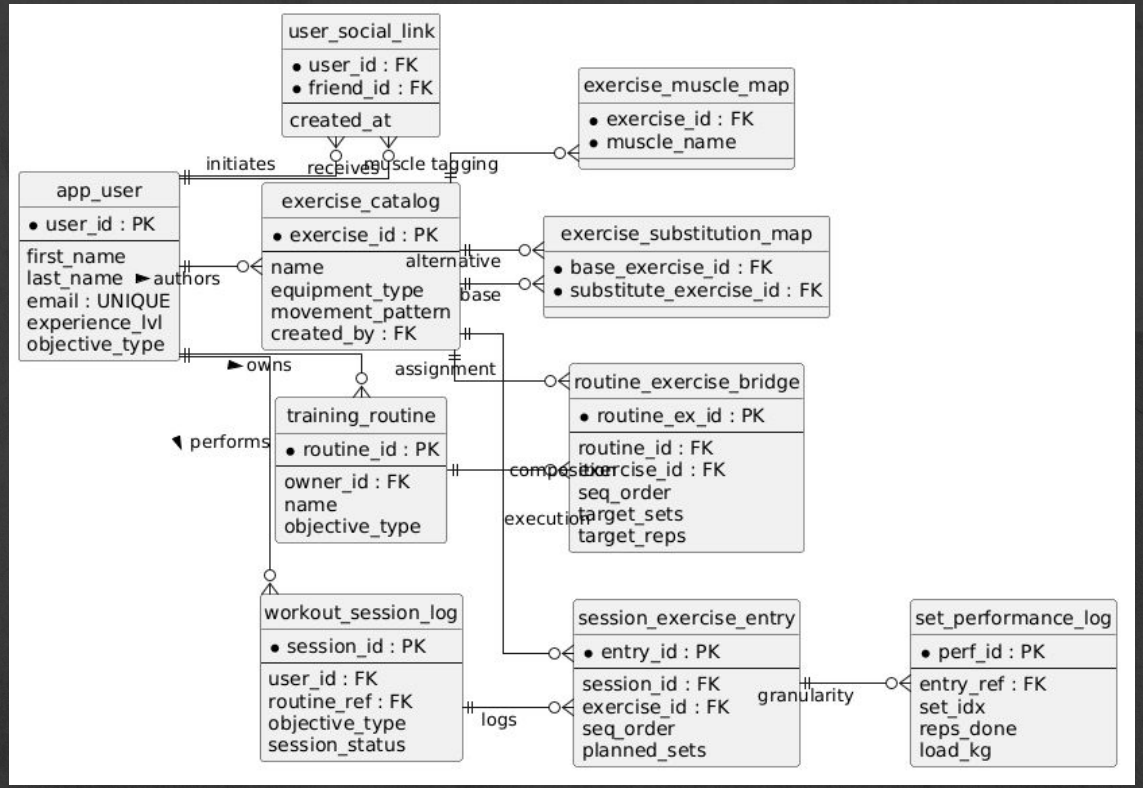
noSQL

Se adapta perfectamente al dominio de la app.

MVP

API y frontend iniciales, el resultado es una app funcional.

¿Sería posible en SQL?



¡GRACIAS!

Frontend

Backend